

--CO3-AT3-Debugging and Optimization



[Secure Exit](#)**SIMATS Online Programming Lab**
JAVA PROGRAMMING

VANGIMALLA BALA SIVA
PRASAD REDDY
192424316RA

QUESTIONS

Q1
ATM Withdrawal – Exception Handling Debugging
10 marks

Q2
Hospital Registration – NullPointer Exception Debugging
10 marks

Q3
E-Commerce Product Search – Array Exception Debugging
10 marks

Q4
Bank Account – Race Condition Debugging
10 marks

Q5
Railway Reservation – Synchronization Debugging
10 marks

5/5 answered

Q1 ATM Withdrawal – Exception Handling Debugging 10 marks

An ATM application is used to process cash withdrawals. The following Java program contains programming errors.

Task:

1. Run the given program and identify the errors.
2. Debug and modify the program.
3. Submit the COMPLETE corrected Java program.
4. The corrected program must handle insufficient balance, zero withdrawal, invalid input, and unexpected errors.
5. Use appropriate try-catch-finally exception handling.
6. The corrected program must execute successfully for all valid test cases.

Faulty Program:

```
import java.util.Scanner;

class ATM {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter account balance: ");
        int balance = sc.nextInt();

        System.out.print("Enter withdrawal amount: ");
        int amount = sc.nextInt();

        try {
```



[Secure Exit](#)**SIMATS Online Programming Lab**
JAVA PROGRAMMING

VANGIMALLA BALA SIVA
PRASAD REDDY
192424316RA

10 marks

5/5 answered

```
        System.out.print("Enter withdrawal amount: ");
        int amount = sc.nextInt();

        try {
            int remaining = balance / amount;

            if (amount > balance) {
                System.out.println("Insufficient balance");
            } else {
                System.out.println("Withdrawal successful");
                System.out.println("Remaining balance = "
                    + (balance - amount));
            }
        }
        catch (ArithmeticException e) {
            System.out.println("Transaction failed");
        }

        System.out.println("Transaction completed");
    }
}
```

Expected correction:

The program must correctly validate the withdrawal amount before performing the transaction, handle division-by-zero and invalid input, and always display "Transaction completed" after exception handling.



Secure Exit

SIMATS Online Programming Lab
 JAVA PROGRAMMING


 VANGIMALLA BALA SIVA
 PRASAD REDDY
 192424316RA

```

1 import java.util.Scanner;
2 import java.util.InputMismatchException;
3 public class ATM{
4     public static void main(String[] args){
5         Scanner sc=new Scanner(System.in);
6         try{
7             System.out.println("Enter account balance: ");
8             int balance=sc.nextInt();
9             System.out.println("Enter withdrawal amount: ");
10            int amount=sc.nextInt();
11            if(amount<=0){
12                throw new ArithmeticException();
13            }
14            if(amount>balance){
15                System.out.println("Insufficient balance");
16            }else{
17                int remaining=balance/amount;
18                System.out.print("Withdrawal successful ");
19                System.out.println("Remaining balance= "+(balance-amount));
20            }
21        }catch(ArithmeticException e){
22            System.out.println("Transaction failed");
23        }catch(InputMismatchException e){
24            System.out.println("Invalid input");
25        }catch(Exception e){
26            System.out.println("Unexpected error");
27        }finally{
28            System.out.println("Registration completed");
29        }
30    }

```

1000
 200

Output

```

Enter account balance:
Enter withdrawal amount:
Withdrawal successful Remaining
balance= 800
Registration completed

```

 Pending manual review



Secure Exit

SIMATS Online Programming Lab
 JAVA PROGRAMMING


 VANGIMALLA BALA SIVA
 PRASAD REDDY
 192424316RA

QUESTIONS

Q1
 ATM Withdrawal – Exception Handling Debugging
 10 marks

Q2
 Hospital Registration – NullPointerException Debugging
 10 marks

Q3
 E-Commerce Product Search – Array Exception Debugging
 10 marks

Q4
 Bank Account – Race Condition Debugging
 10 marks

Q5
 Railway Reservation – Synchronization Debugging
 10 marks

5/5 answered

Q2 Hospital Registration – NullPointerException Debugging

10 marks

A hospital patient registration application crashes when patient information is unavailable.

Debug the following complete Java program.

Task:

1. Identify the runtime error.
2. Modify the program to handle a missing patient name.
3. Handle invalid age input.
4. Use try-catch-finally appropriately.
5. The program must always display "Registration completed".
6. Submit the COMPLETE corrected Java program.

Faulty Program:

```

import java.util.Scanner;

class PatientRegistration {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter patient name: ");
        String inputName = sc.nextLine();

        String name;

        if (inputName.equals("NULL")) {

```



Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

VANGIMALLA BALA SIVA
PRASAD REDDY
192424316RA

5/5 answered

```
String name;

if (inputName.equals("NULL")) {
    name = null;
} else {
    name = inputName;
}

System.out.print("Enter age: ");
int age = sc.nextInt();

System.out.println("Patient Name: "
    + name.toUpperCase());

if (age >= 18) {
    System.out.println("Adult patient");
} else {
    System.out.println("Minor patient");
}

System.out.println("Registration completed");
}
}
```

The corrected program must prevent NullPointerException and handle invalid age input without terminating unexpectedly.



Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

VANGIMALLA BALA SIVA
PRASAD REDDY
192424316RA

Your Fixed Code

```
1 import java.util.Scanner;
2 class PatientRegistration{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         try{
6             System.out.print("Enter patient name: ");
7             String inputName=sc.nextLine();
8             String name;
9             int age=sc.nextInt();
10            if(inputName.equals("NULL")){
11                name="Unknown";
12            }else{
13                name=inputName;
14            }
15            System.out.println("Enter age: ");
16            System.out.println("Patient Name: "+name.toUpperCase());
17            if(age>=18){
18                System.out.println("Adult patient");
19            }else if(age>=0){
20                System.out.println("Minor patient");
21            }else{
22                System.out.println("Invalid age");
23            }
24        }catch(Exception e){
25            System.out.println("Invalid input");
26        }finally{
27            System.out.println("Registration completed");
28        }
29    }
30 }
```

Test Input

Arun
35

Output

Enter patient name: Enter age:
Patient Name: ARUN
Adult patient
Registration completed

Pending manual review



Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

VANGIMALLA BALA SIVA
PRASAD REDDY
192424316RA

QUESTIONS

Q1
ATM Withdrawal – Exception Handling Debugging
10 marks

Q2
Hospital Registration – NullPointerException Debugging
10 marks

Q3
E-Commerce Product Search – Array Exception Debugging
10 marks

Q4
Bank Account – Race Condition Debugging
10 marks

Q5
Railway Reservation – Synchronization Debugging
10 marks

5/5 answered

Q3 E-Commerce Product Search – Array Exception Debugging 10 marks

An online shopping application allows customers to select a product using a product number from 1 to 5.

The following program contains an array indexing error.

Task:

1. Identify the error.
2. Correct the product-number and array-index logic.
3. Handle invalid product numbers.
4. Handle non-numeric input.
5. Submit the COMPLETE corrected Java program.
6. The program must display "Search completed" after processing.

Faulty Program:

```
import java.util.Scanner;

class ProductSearch {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        String[] products = {
            "Laptop",
            "Mobile",
            "Tablet",
            "Headphone".
        }
```



Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

VANGIMALLA BALA SIVA
PRASAD REDDY
192424316RA

5/5 answered

```
"Laptop",
"Mobile",
"Tablet",
"Headphone",
"Keyboard"
};

System.out.print("Enter product number (1-5): ");
int productNumber = sc.nextInt();

System.out.println("Selected product: "
+ products[productNumber]);

System.out.println("Search completed");
}
}
```

Correct the program so that product numbers 1, 2, 3, 4 and 5 correctly access the corresponding products.

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 import java.util.Scanner;
2 class ProductSearch{
3     public static void main(String[] args) {
4         Scanner sc=new Scanner(System.in);
5         String[] products={
```

Test Input

1


SIMATS
ENGINEERING

Secure Exit

SIMATS Online Programming Lab
 JAVA PROGRAMMING


 VANGIMALLA BALA SIVA
 PRASAD REDDY
 192424316RA

Your Fixed Code

```

1 import java.util.Scanner;
2 class ProductSearch{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         String[] products={
6             "Laptop",
7             "Mobile",
8             "Headphones",
9             "Keyboard"
10        };
11        try{
12            int productNumber=sc.nextInt();
13            System.out.println("Selected product: "+products[productNumber]);
14        }catch(ArrayIndexOutOfBoundsException e){
15            System.out.println("Invalid product number");
16        }finally{
17            System.out.println("Search completed");
18        }
19    }
20 }
21 }

```

Submitted

Test Input

1

Output

Selected product: Laptop
Search completed

Pending manual review


SIMATS
ENGINEERING

Secure Exit

SIMATS Online Programming Lab
 JAVA PROGRAMMING


 VANGIMALLA BALA SIVA
 PRASAD REDDY
 192424316RA

QUESTIONS

Q1

ATM Withdrawal – Exception Handling Debugging

10 marks

Q2

Hospital Registration – NullPointerException Debugging

10 marks

Q3

E-Commerce Product Search – Array Exception Debugging

10 marks

Q4

Bank Account – Race Condition Debugging

10 marks

Q5

Railway Reservation – Synchronization Debugging

10 marks

5/5 answered

Q4 Bank Account – Race Condition Debugging

10 marks

A bank account contains ₹10,000. Two ATM threads simultaneously attempt to withdraw ₹7,000 and ₹6,000.

The following program contains a race condition.

Task:

1. Identify the race condition.
2. Identify the critical section.
3. Explain why both threads may pass the balance check.
4. Modify the complete program using synchronized.
5. Ensure that the balance can never become negative.
6. Submit the COMPLETE corrected Java program.

Faulty Program:

```

class BankAccount {
    int balance = 10000;

    void withdraw(int amount) {
        if (balance >= amount) {
            System.out.println(
                Thread.currentThread().getName()
                + " withdrawing " + amount);
        }
    }
}

```

5/5 answered

```
+ " withdrawing " + amount);

try {
    Thread.sleep(100);
}
catch (InterruptedException e) {
    System.out.println("Interrupted");
}

balance = balance - amount;

System.out.println(
    Thread.currentThread().getName()
    + " withdrawal successful");
}
else {
    System.out.println(
        Thread.currentThread().getName()
        + " insufficient balance");
    }
}
}

class ATM extends Thread {

    BankAccount account;
    int amount;
```

```
this.account = account;
this.amount = amount;
}

public void run() {
    account.withdraw(amount);
}
}

class BankApplication {

    public static void main(String[] args)
    throws InterruptedException {

        BankAccount account = new BankAccount();

        ATM atm1 = new ATM(account, 7000);
        ATM atm2 = new ATM(account, 6000);

        atm1.setName("ATM-1");
        atm2.setName("ATM-2");

        atm1.start();
        atm2.start();

        atm1.join();
        atm2.join();

        System.out.println(
            "Final Balance = " + account.balance);
```

```
System.out.println(
    "Final Balance = " + account.balance);
}
}
```

Correct the program so that only one withdrawal succeeds.

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 class BankApplication{
2     public static void main(String[] args) throws InterruptedException{
3         BankAccount account=new BankAccount();
4         ATM atm1=new ATM(account,7000);
5         ATM atm2=new ATM(account,6000);
6         atm1.setName("ATM-1 ");
7         atm2.setName("ATM-2 ");
8         atm1.start();
9         atm2.start();
10        atm1.join();
11        atm2.join();
12        System.out.println("Final Balance: "+account.balance);
13    }
14 }
15 class BankAccount{
16     int balance=10000;
```

Test Input

stdin input (optional)

Output

```
ATM-1 Withdrawing: 7000
ATM-1 Withdrawal successful
ATM-2 Insufficient balance
Final Balance: 3000
```

 Pending manual review

```
16     int balance=10000;
17     synchronized void withdraw(int amount){
18         if(balance>=amount){
19             System.out.println(Thread.currentThread().getName()+"Wit
20             try{
21                 Thread.sleep(100);
22             }catch(InterruptedException e){
23             }
24             balance=balance-amount;
25             System.out.println(Thread.currentThread().getName()+"Wit
26         }else{
27             System.out.println(Thread.currentThread().getName()+"Ins
28         }
29     }
30 }
31 }
32 class ATM extends Thread{
33     BankAccount account;
34     int amount;
35     ATM(BankAccount account,int amount){
36         this.account=account;
37         this.amount=amount;
38     }
39 }
40 public void run(){
41     account.withdraw(amount);
42 }
43 }
44 //class BankApplication{
```



SIMATS
ENGINEERING

Secure Exit

SIMATS Online Programming Lab
 JAVA PROGRAMMING


 VANGIMALLA BALA SIVA
 PRASAD REDDY
 192424316RA

QUESTIONS

Q1 ✓
 ATM Withdrawal – Exception Handling Debugging
 10 marks

Q2 ✓
 Hospital Registration – NullPointerException Debugging
 10 marks

Q3 ✓
 E-Commerce Product Search – Array Exception Debugging
 10 marks

Q4 ✓
 Bank Account – Race Condition Debugging
 10 marks

Q5 ✓
 Railway Reservation – Synchronization Debugging
 10 marks

5/5 answered

Q5 Railway Reservation – Synchronization Debugging 10 marks

A railway reservation system has only one available seat. Two customers attempt to reserve the seat simultaneously.

The following complete Java program contains a synchronization problem.

Task:

1. Identify the race condition.
2. Debug the program using the synchronized keyword.
3. Ensure that only ONE customer can reserve the available seat.
4. The second customer must receive "No seat available".
5. Submit the COMPLETE corrected Java program.

Faulty Program:

```

class Reservation {
    int seats = 1;

    void bookSeat(String customer) {
        if (seats > 0) {
            System.out.println(customer
                + " is booking a seat");
            try {
                Thread.sleep(100);
            }
        }
    }
}
    
```


SIMATS
ENGINEERING

Secure Exit

SIMATS Online Programming Lab
 JAVA PROGRAMMING


 VANGIMALLA BALA SIVA
 PRASAD REDDY
 192424316RA

5/5 answered

```

        try {
            Thread.sleep(100);
        }
        catch (InterruptedException e) {
            System.out.println("Thread interrupted");
        }

        seats--;

        System.out.println(customer
            + " booking successful");

    } else {
        System.out.println(customer
            + " - No seat available");
    }
}

class Customer extends Thread {
    Reservation reservation;
    String customerName;

    Customer(Reservation reservation, String customerName) {
        this.reservation = reservation;
        this.customerName = customerName;
    }
}
    
```



SIMATS
ENGINEERING

Secure Exit

SIMATS Online Programming Lab
 JAVA PROGRAMMING


 VANGIMALLA BALA SIVA
PRASAD REDDY
192424316RA

```

        int customerName = customerName;
    }

    public void run() {
        reservation.bookSeat(customerName);
    }
}

class RailwayReservation {

    public static void main(String[] args) {

        Reservation reservation = new Reservation();

        Customer c1 =
            new Customer(reservation, "Customer 1");

        Customer c2 =
            new Customer(reservation, "Customer 2");

        c1.start();
        c2.start();
    }
}

The corrected program must protect the shared seat resource using synchronization.

```

 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.


SIMATS
ENGINEERING

Secure Exit

SIMATS Online Programming Lab
 JAVA PROGRAMMING


 VANGIMALLA BALA SIVA
PRASAD REDDY
192424316RA

Your Fixed Code

```

1 class RailwayReservation{
2     public static void main(String[] args) throws InterruptedException{
3         Reservation r=new Reservation();
4         Customer c1=new Customer(r,"Customer 1 ");
5         Customer c2=new Customer(r,"Customer 2 ");
6         c1.start();
7         c2.start();
8         c1.join();
9         c2.join();
10    }
11 }
12 class Reservation{
13     int seats=1;
14     synchronized void bookSeat(String name){
15         if(seats>0){
16             System.out.println(name+"is booking a seat");
17             try{
18                 Thread.sleep(100);
19             }catch(InterruptedException e){
20
21             }
22             seats--;
23             System.out.println(name+"booked the seat");
24         }else{
25             System.out.println("No seat available");
26         }
27     }
28 }

```

Test Input
 stdin input (optional)

Output

```

Customer 1 is booking a seat
Customer 1 booked the seat
No seat available

```

 Pending manual review

```
14 synchronized void bookSeat(String name){
15     if(seats>0){
16         System.out.println(name+"is booking a seat");
17         try{
18             Thread.sleep(100);
19         }catch(InterruptedException e){
20
21         }
22         seats--;
23         System.out.println(name+"booked the seat");
24     }else{
25         System.out.println("No seat available");
26     }
27 }
28 }
29 class Customer extends Thread{
30     Reservation r;
31     String name;
32     Customer(Reservation r,String name){
33         this.r=r;
34         this.name=name;
35     }
36     public void run(){
37         r.bookSeat(name);
38     }
39 }
```

 Pending manual review Submitted